Advanced SQL Exercises for Online Retail Store

Exercise 1: Ranking and Window Functions

Goal: Use ROW_NUMBER(), RANK(), DENSE_RANK(), OVER(), and PARTITION BY.

Scenario:

Find the top 3 most expensive products in each category using different ranking functions.

Steps:

1. Use ROW_NUMBER() to assign a unique rank within each category.
Solution:

```
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    ROW_NUMBER() OVER
    (
        PARTITION BY Category
        ORDER BY Price DESC
    ) AS RowNum
FROM Products;
```

2. Use RANK() and DENSE_RANK() to compare how ties are handled.

Solution:

```
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    RANK() OVER
    (
        PARTITION BY Category
        ORDER BY Price DESC
    ) AS RankNum
FROM Products;


SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    DENSE_RANK() OVER
```

```
    (
        PARTITION BY Category
        ORDER BY Price DESC
    ) AS DenseRankNum
FROM Products;
```

3. Use PARTITION BY Category and ORDER BY Price DESC.

```
WITH ProductRanks AS
(
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        ROW_NUMBER() OVER
        (
            PARTITION BY Category
            ORDER BY Price DESC
        ) AS RowNum
    FROM Products
)

SELECT *
FROM ProductRanks
WHERE RowNum <= 3;
```

Exercise 2: Aggregation with GROUPING SETS, CUBE, and ROLLUP

Goal: Analyze sales data across multiple dimensions.

Scenario:

Generate a report showing total quantity sold by Region and Category using GROUPING SETS, ROLLUP, an

Steps:

1. Join Orders, OrderDetails, Customers, and Products.
2. Use GROUPING SETS to get totals by Region, Category, and both.

Solution:

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
```

```
JOIN Customers c
   ON o.CustomerID = c.CustomerID
JOIN OrderDetails od
   ON o.OrderID = od.OrderID
JOIN Products p
   ON od.ProductID = p.ProductID
GROUP BY GROUPING SETS
(
   (c.Region, p.Category),
   (c.Region),
   (p.Category)
);
```

3. Use ROLLUP to get subtotals and grand totals.

```
SELECT
   c.Region,
   p.Category,
   SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN Customers c
   ON o.CustomerID = c.CustomerID
JOIN OrderDetails od
   ON o.OrderID = od.OrderID
JOIN Products p
   ON od.ProductID = p.ProductID
GROUP BY ROLLUP
(
   c.Region,
   p.Category
);
```

4. Use CUBE to get all combinations of Region and Category.

```
SELECT
   c.Region,
   p.Category,
   SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN Customers c
   ON o.CustomerID = c.CustomerID
JOIN OrderDetails od
```

```
    ON o.OrderID = od.OrderID
JOIN Products p
    ON od.ProductID = p.ProductID
GROUP BY CUBE
(
    c.Region,
    p.Category
);
```

Exercise 3: CTEs and MERGE

Goal: Use WITH, CTEs, Recursive CTEs, and MERGE.

Scenario:

a) Create a recursive CTE to generate a calendar table.

b) Use a MERGE statement to update or insert product prices from a staging table.

Steps:

1. Create a recursive CTE to generate dates from '2025-01-01' to '2025-01-31'.

```
WITH Calendar AS
(
    SELECT CAST('2025-01-01' AS DATE) AS DateValue

    UNION ALL

    SELECT DATEADD(DAY,1,DateValue)
    FROM Calendar
    WHERE DateValue < '2025-01-31'
)

SELECT *
FROM Calendar
OPTION (MAXRECURSION 100);
```

2. Create a StagingProducts table with updated prices.

```
CREATE TABLE StagingProducts
(
    ProductID INT,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2)
);
```

3. Use MERGE to update existing products or insert new ones.

```
INSERT INTO StagingProducts
VALUES
```

```
(1,'Laptop','Electronics',65000),
(2,'Mouse','Electronics',700),
(10,'Printer','Electronics',12000);


MERGE Products AS Target
USING StagingProducts AS Source
ON Target.ProductID = Source.ProductID

WHEN MATCHED THEN
   UPDATE SET
      Target.Price = Source.Price

WHEN NOT MATCHED THEN
   INSERT
   (
      ProductID,
      ProductName,
      Category,
      Price
   )
   VALUES
   (
      Source.ProductID,
      Source.ProductName,
      Source.Category,
      Source.Price
   );
```

Exercise 4: PIVOT and UNPIVOT

Goal: Transform data for reporting.

Scenario:

Show monthly sales quantity per product in a pivoted format, and then unpivot it back.

Steps:

1. Aggregate sales by Product and Month.
Solution:

```
SELECT
    p.ProductName,
    MONTH(o.OrderDate) AS SalesMonth,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN OrderDetails od
    ON o.OrderID = od.OrderID
JOIN Products p
    ON od.ProductID = p.ProductID
GROUP BY
    p.ProductName,
    MONTH(o.OrderDate);
```

2. Use PIVOT to convert rows into columns (one column per month).

```
SELECT *
FROM
(
    SELECT
        p.ProductName,
        MONTH(o.OrderDate) AS SalesMonth,
        od.Quantity
    FROM Orders o
    JOIN OrderDetails od
        ON o.OrderID = od.OrderID
    JOIN Products p
        ON od.ProductID = p.ProductID
) AS SourceTable

PIVOT
(
    SUM(Quantity)
    FOR SalesMonth IN
    (
        [1],[2],[3],[4],[5],[6],
        [7],[8],[9],[10],[11],[12]
```

```
    )
) AS PivotTable;
```

3. Use UNPIVOT to convert the pivoted data back into row format.

```
SELECT
    ProductName,
    SalesMonth,
    Quantity
FROM PivotTable

UNPIVOT
(
    Quantity
    FOR SalesMonth IN
    (
        [1],[2],[3],[4],[5],[6],
        [7],[8],[9],[10],[11],[12]
    )
) AS UnpivotTable;
```

Exercise 5: Using CTE to Simplify a Query

Goal: Use Common Table Expressions (CTEs) to simplify complex queries.

Scenario:

The business wants to find all customers who have placed more than 3 orders in total.

Steps:

1. Create a CTE that counts the number of orders placed by each customer.
2. Select only those customers who have placed more than 3 orders.

Sample Query:

```
WITH CustomerOrderCounts AS
    ( SELECT
        o.CustomerID,
        COUNT(o.OrderID) AS OrderCount
    FROM Orders o
    GROUP BY o.CustomerID
)
SELECT
    c.CustomerID,
    c.Name,
    coc.OrderCount
```

```
FROM CustomerOrderCounts coc
JOIN Customers c ON c.CustomerID = coc.CustomerID
WHERE coc.OrderCount > 3;
```

Solution:

```
WITH CustomerOrderCounts AS
(
    SELECT
        CustomerID,
        COUNT(OrderID) AS OrderCount
    FROM Orders
    GROUP BY CustomerID
)

SELECT
    c.CustomerID,
    c.Name,
    coc.OrderCount
FROM CustomerOrderCounts coc
JOIN Customers c
    ON coc.CustomerID = c.CustomerID
WHERE coc.OrderCount > 3;
```